

RESEARCH ARTICLE

WILEY

A web-based integrated GUI for 3D modeling, kinematic study, and control of robotic manipulators

Ujjal Dey  | Kumar Cheruvu S.

Department of Mechanical Engineering,
Indian Institute of Technology Kharagpur,
Kharagpur, India

Correspondence

Ujjal Dey, Department of Mechanical
Engineering, Indian Institute of
Technology Kharagpur, Kharagpur
721302, India.

Email: ujjal.dey@iitkgp.ac.in

Abstract

The simulation-based analysis of any robotic system has emerged broadly in the past few years for application in educational and research enhancement. Currently, many simulation tools are available for different research areas on robotics with some advantages and limitations. In this study, the methodology of creating a web-based graphical user interface is proposed for motion simulations and kinematic study of industrial robotic manipulators. It describes simple and straightforward modeling techniques using Web Graphics Library for creating any three-dimensional virtual model of the robots within an HTML canvas for its motion simulation and different kinds of analysis. The positional kinematic study of the programmable universal machine for assembly 560 (PUMA 560) industrial robot through the developed interface is presented here, which helps in understanding the whole framework and can be implemented quickly for any other simulation system development. An essential aspect of the kinematic study of these serial chain robots is inverse kinematic analysis demonstrated with PUMA 560 robot with all the possible solutions. Finally, the interface is connected to an actual system following web-based communication architecture for remote control of the actual PUMA robot.

KEYWORDS

interface, modeling, PUMA, simulation, Web, WebGL

1 | INTRODUCTION

Robotics is a wide field of education and research related to designing, control, and artificial intelligence with applications in different domains connecting multiple disciplines of engineering science [28]. Whatever be the application, the ultimate goal of any robotic system is achieved through first its motion understanding. Therefore, the study of robot kinematics is essential for any research or educational purpose where simulation models help a lot. This type of simulation model provides basic familiarization about the robotic system, which is required for understanding engineering courses in

robotics and also for research-level work. The setup of the simulation model can be linked with an actual robot for its remote control and validations of any programming experiments. Now for the analysis of the robot motion or to perform any other test from the simulation interface, it always requires a schematic or three-dimensional (3D) virtual model representation for proper visualization. The most common and frequently useable toolbox for this type of robotic simulation is the MATLAB software [6,7]. MATLAB is mainly used for an academic purpose, which allows the construction of interfaces and testing of any program for the desired application [15]. In this process, the 3D computer-aided design (CAD) model is designed

through some 3D geometric modeling software, and its mathematical modeling and analysis is presented through the MATLAB/Simulink or LabVIEW-based environment [1,5,14,19,24,39,46]. A major limitation arises on the use of MATLAB-based applications due to its dependency on the user system hardware configuration. Multiple other tools like Gazebo, V-Rep, ROS, RoboAnalyzer, Blender, RoboDK, and so forth, are also available for 3D modeling, simulation, and programming of different types of robotic systems [18,32,34–36]. A detailed survey based on user feedback has been done on the familiar available tools for robotic simulation [20]. Most of the tools are explored as educational purposes binding them in the form of a user interface for its application [16,17,48]. Apart from its usage through a particular system, the primary need for these tools arises through any network-based education process.

The current trend of web-based education systems through the internet is gaining popularity due to its easy implementation and accessibility worldwide [22,27]. An essential requirement of this web technology for robotics courses is the proper onsite availability of the developed tools for performing the experiments. The experimental interface developed using any proprietary software needs to be either installed on the end-user's system or on the server end to compile the simulations for performing any analysis. Studies suggest that, in many situations, it puts a limitation on continuous and frequent access for the users. Server-dependent simulation program also creates a lag between execution and actual visualization. As the primary end-user of this type of simulation tools are mostly engineering students, a continuous upgradation is going on to remove any software or system dependencies. Therefore, to easily integrate into a web browser, a Java 3D application-based platform was implemented for robotics simulations development [21,25]. It is an open-source interactive tool for building 3D graphics with multiple libraries. However, it still requires an installation of the Java application plugin on the user system for running the simulation. Much research efforts are being put up nowadays for developing independent application platforms for accessibility from any device. For better enrichment of 3D display graphics within the interface, Open Graphics Library- and Web Graphics Library (WebGL)-based platforms are emerging [26,45,47]. A WebGL-based interface for 3D modeling and kinematic study of robotic manipulators has the additional advantage of web browser-based running capability, which increases the accessibility up to a large extent. This type of interface visualizes 3D simulation with an interactive application enabled, and it is free from any supporting software or plugins. It has better graphics and has an additive impact due to its light version. Web-based

simulation modules are system independent, and the source code is freely available for the improvement of the program.

In this study, first, the basic methodology of building any web-compatible graphical user interface (GUI) with different 3D modeling techniques using WebGL is described. The interface is then prepared for the kinematic study of the programmable universal machine for assembly 560 (PUMA 560) industrial robot with the 3D motion simulation visualization. A brief introductory work on this PUMA 560 robot for educational enhancement through virtual reality has been presented in earlier work [12]. The current article has been extended with further detailed analysis of the kinematics study of these robotic manipulators with different 3D geometric modeling procedures in a web-based platform. It also explains the building procedure of an integrated GUI in HTML5 canvas. An essential aspect of the kinematic study of these types of serial link manipulator is inverse kinematics analysis, which describes different possible solutions of the robot for reaching a particular position with orientation in 3D Cartesian space. This form of multiple solution visualization demonstrated with PUMA robot within the same scene is a unique approach. Here, a user can obtain all the possible inverse kinematics solutions and can visualize each configuration individually. The simulation model is then connected with an actual system for controlling the PUMA robot from the prepared interface following a serial communication procedure. A user can control the actual robot joint movement from the control panel created in the interface with direct visual feedback through an Internet protocol (IP) camera.

1.1 | Application in robotics virtual labs development

The main focus of the work described here is to develop a web-based integrated environment that provides a platform for creating any simulation system with 3D visualization along with real-time control. It describes a unique approach of exploring the WebGL technology for 3D modeling of robots. This type of simulation platform is of a lighter version that easily renders the 3D computer graphics. The prime application of this type of web-based platform is to enhance the E-learning technology through a virtual laboratory program. The concept of a virtual laboratory for engineering education has been proposed earlier and has been implemented in different fields of study with various techniques [2,9]. These types of virtual laboratories are mostly required for robotics education due to the unavailability of expensive hardware in many institutions [3,37]. Here, as the entire content is in web

format, experiments are easily accessed globally through a webserver that does not require any supporting application in either client-side or server-side. Simulations can run from anywhere without using any plugin even without continuous communication with the global server. The framework shown here can be applied to any other discipline research work apart from robotics. This type of 3D simulations helps in a better understanding of the physical system with proper visualizations, which add an advantage in the learning process.

2 | FORWARD AND INVERSE KINEMATICS OF THE PUMA 560 INDUSTRIAL ROBOTIC MANIPULATOR

Kinematics is the study of motion characteristics of the subjects without considering the forces or torques that cause the motion. Within the scope of the study, position, velocity, and acceleration analyses of any rigid body are done. In robotics under positional kinematics, two significant aspects of studies are the forward kinematics and inverse kinematics formulation of any serial or parallel link robotic manipulators. A serial link manipulator is an open-chain linkage consisting of an end-effector connected to the base, whereas a parallel manipulator is a closed chain mechanism composed of a moving platform connected to the fixed platform used mostly for precise motion [42]. A brief mathematical aspect of the PUMA 560 robot is described here as it is a familiar industrial robot and a detailed literature is available in any standard robotic textbook [8,29]. The Unimation PUMA 560 is a 6 degrees of freedom (DOF) serial link manipulator with all rotary joints. Apart from its industrial application, it is mostly used as an educational robot in many institutions for teaching robotics courses [4]. Along with PUMA 560, which is a 6-DOF serial link robot, other manipulators like cyton, Movemaster RM501, and so forth, with 7 and 5 DOFs, respectively, are also used in the educational environment. They help in understanding the concepts like underactuated and overactuated cases in robotics.

The forward kinematics problem deals with the mapping of parameters from joint space to cartesian space, which determines the robot end-point position and orientation for a specified set of joint angular values. The kinematic analysis is done by attaching the coordinate frames to the robot joints following the modified Denavit-Hartenberg D-H convention [10]. The transformation between any two frames is represented by a homogeneous matrix expressed in terms of four fundamental quantities associated with joints i and j . These four parameters are called D-H parameters, which are

TABLE 1 D-H parameters of PUMA 560

Link i	D-H parameters			
	α_{i-1}	a_{i-1} (mm)	d_i (mm)	θ_i
1	0	0	0	θ_1
2	-90	0	0	θ_2
3	0	a_2	d_3	θ_3
4	-90	a_3	d_4	θ_4
5	90	0	0	θ_5
6	-90	0	0	θ_6

Abbreviations: D-H, Denavit-Hartenberg; PUMA, programmable universal machine for assembly.

link length (a), link twist (α), link offset (d), and joint angle (θ), respectively. The modified D-H parameters of the PUMA 560 robot are given in Table 1. The regular expression of each homogeneous transformation matrix of the i^{th} frame with respect to the $i-1^{\text{th}}$ frame is expressed in terms of these four quantities, which is given by

$${}^{i-1}T_i = \begin{bmatrix} C\theta_i & -S\theta_i & 0 & a_{i-1} \\ S\theta_i C\alpha_{i-1} & C\theta_i C\alpha_{i-1} & -S\alpha_{i-1} & -S\alpha_{i-1}d_i \\ S\theta_i S\alpha_{i-1} & C\theta_i S\alpha_{i-1} & C\alpha_{i-1} & C\alpha_{i-1}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

where C and S are the cosine and sine of the respective joint parameters.

From the forward kinematics analysis, the position and orientation of the manipulator end-effector from the base frame are obtained by matrix multiplication of six transformation matrix

$${}^0T_6 = {}^0T_1 {}^1T_2 {}^2T_3 {}^3T_4 {}^4T_5 {}^5T_6 \quad (2)$$

$$= \begin{bmatrix} r_{11} & r_{12} & r_{13} & P_x \\ r_{21} & r_{22} & r_{23} & P_y \\ r_{31} & r_{32} & r_{33} & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forward kinematics formulation is straightforward, which always has a solution for given joint inputs. The inverse problem is much more difficult because there may be no systematic closed-form solution always. Inverse kinematics formulation provides joint angles information for a given desired position and orientation information of the manipulator end-effector. As the kinematic equations of the manipulator are nonlinear, there will be multiple solutions for the existing set of solutions. Now to solve the equation apart from numerical solution, closed-form solutions can be obtained by

either algebraic or geometric methods. From Equation (1), we have

$$\begin{bmatrix} r_{11} & r_{12} & r_{13} & P_x \\ r_{21} & r_{22} & r_{23} & P_y \\ r_{31} & r_{32} & r_{33} & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = [{}^0T_1(\theta_1)][{}^1T_2(\theta_2)][{}^2T_3(\theta_3)][{}^3T_4(\theta_4)] \\ [{}^4T_5(\theta_5)][{}^5T_6(\theta_6)]. \quad (3)$$

For finding the inverse kinematics solutions algebraically, a simple technique is used to separate each variable and express it in terms of known parameters. Equation (3) is multiplied by the inverse of the matrix ${}^0T_1[\theta_1]$, we have

$$\begin{bmatrix} C_1 & S_1 & 0 & 0 \\ -C_1 & S_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & P_x \\ r_{21} & r_{22} & r_{23} & P_y \\ r_{31} & r_{32} & r_{33} & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = [{}^1T_2(\theta_2)][{}^2T_3(\theta_3)] \\ [{}^3T_4(\theta_4)][{}^4T_5(\theta_5)][{}^5T_6(\theta_6)] = {}^1T_6. \quad (4)$$

After matrix multiplication equating the elements 14 and 34 of both side of Equation (4), the first joint variable θ_1 can be expressed as

$$\theta_1 = A \tan 2(p_y, p_x) - A \tan 2\left(d_3, \pm\sqrt{p_y^2 + p_x^2 - d_3^2}\right). \quad (5)$$

${}^0T_1[\theta_1]$ is found out from Equation (1) and using the D-H parameters of link 1. Depending on the plus or minus sign of Equation (5), there are two possible solutions for θ_1 . Similarly, other solutions can also be found out, which are expressed as follows [8]:

$$\theta_2 = A \tan 2[(-a_3 - a_2 C_3)p_z - (C_1 p_x + S_1 p_y)(d_4 - a_2 S_3), (-a_3 - a_2 C_3)p_z + (-a_3 - a_2 C_3)p_z] \\ - A \tan 2(a_3, d_4) \\ - A \tan 2\left(K, \pm\sqrt{a_3^2 + d_4^2 - K^2}\right), \quad (6)$$

$$\theta_3 = A \tan 2(a_3, d_4) \\ - A \tan 2\left(K, \pm\sqrt{a_3^2 + d_4^2 - K^2}\right), \quad (7)$$

where

$$K = (P_x^2 + P_y^2 + P_z^2 - a_2^2 - a_3^2 - d_3^2 - d_4^2)/2a_2^2, \quad (8)$$

$$\theta_4 = A \tan 2(-r_{13}S_1 + r_{23}C_1, -r_{13}C_1C_{23} - r_{23}S_1C_{23} + r_{33}S_{23}), \quad (9)$$

$$\theta_5 = A \tan 2(S_5, C_5), \quad (10)$$

where

$$S_5 = -r_{13}(C_1C_{23}C_4 + S_1S_4) - r_{23}(S_1C_{23}C_4 - C_1S_4) + r_{23}S_{23}C_4, \quad (11)$$

$$C_5 = r_{13}(-C_1S_{23}) + r_{23}(-S_1S_{23}) - r_{23}C_{23}, \quad (12)$$

$$\theta_6 = A \tan 2(S_6, C_6), \quad (13)$$

where

$$S_6 = -r_{11}(C_1C_{23}C_4 - S_1C_4) - r_{21}(S_1C_{23}S_4 + C_1C_4) + r_{31}S_{23}S_4, \quad (14)$$

$$C_6 = r_{11}[(C_1C_{23}C_4 + S_1S_4)C_5 - C_1S_{23}S_5] + r_{21}[(S_1C_{23}C_4 - C_1S_4)C_5 - S_1S_{23}S_5] - r_{31}(S_{23}C_5C_4 - C_{23}S_5). \quad (15)$$

3 | WEB INTERFACE FOR 3D MODELING AND SIMULATION USING WebGL

Web-based platforms are widely used nowadays for interactive 3D model-based visual simulation in the different fields of engineering applications. WebGL is a JavaScript application programming interface for rendering 2D or 3D graphics within any HTML5-supported web browser [41]. It uses the HTML5 canvas element and is accessed using the document object model interfaces. The WebGL-renderer module allows creating any 2D or 3D graphics within the canvas created without the use of any plugins, which is visualized in any standard HTML5 compatible web browsers. Simple geometries are easily created by defining the geometrical vertex point's data with specific commands to build any 3D virtual model. For complex 3D components, the initial designing part is done using CAD software's like Solidworks, CATIA, Autodesk, Creo, and so forth, and then, the extracted triangular vertex coordinate data are incorporated into the WebGL program [23]. Application of direct use of WebGL program seems to be a fine approach, but it leads to difficulty when a 3D structure with multiple components needs to be rendered. For the visualization of a single element, one needs to specify all the vertex coordinates data points accurately, which have to be

extracted from the CAD model using some tool or algorithm [40]. Therefore, the direct use of the WebGL program is quite cumbersome, where using a library reduces the steps and provides a more natural way to create 3D geometric models. Currently, many JavaScript libraries like Three.js, SceneJS, BabylonJS use WebGL and provide many high-level features for 3D object visualization. Three.js is a high-level utility library that makes WebGL program much easier and more straightforward. It is an opens source JavaScript 3D library that uses WebGL technology and provides multiple features of camera control, light control, and animations [13,33]. As it uses JavaScript coding language, it can be combined with other libraries, which are capable of giving interactive applications. Implementation of this technology for various web-based experiments is going on in many fields of study [38,43]. Here, the library of Three.js is used for 3D modeling and particular scene visualization purposes. Now depending on the complexity level of the 3D model to be created, different methodologies can be applied, which are discussed in the next subsection.

3.1 | 3D Modeling from basic shapes/geometries

The 3D modeling procedure through the web environment first requires some initial set of tasks to be done. The very first step for viewing a 3D model in a web browser is to create an HTML5 canvas within which the scene is defined using THREE. Scene () function and a camera are set specifying its properties. The scene acts as a container that holds all the objects to be rendered. Then, the WebGL-renderer module is called through THREE.WebGLRenderer () function, which renders the scene and camera. The following sequential steps need to be followed to create any standard 3D shape using the three.js library and to visualize it within the canvas, as shown in Figure 1. For modeling any 3D structure, the first step is creating the single building block geometry of the 3D model. The geometry of the object is defined by specifying its dimensions, and then, the material is added. It acts as a wrapper over the geometry created. Next, as shown in Figure 1 by the directional arrow, both the geometry and the material are then combined in a form called mesh, which is visualized when added to the 3D scene created within the HTML canvas. As shown in Figure 1, this mesh can be visually enhanced by adding color properties or any kind of texture in the material part. Further light sources are added to the scene for better visualization or any required lightning animation effect. Three.js library has a large set of functions predefined for creating basic 3D shapes of standard geometries.

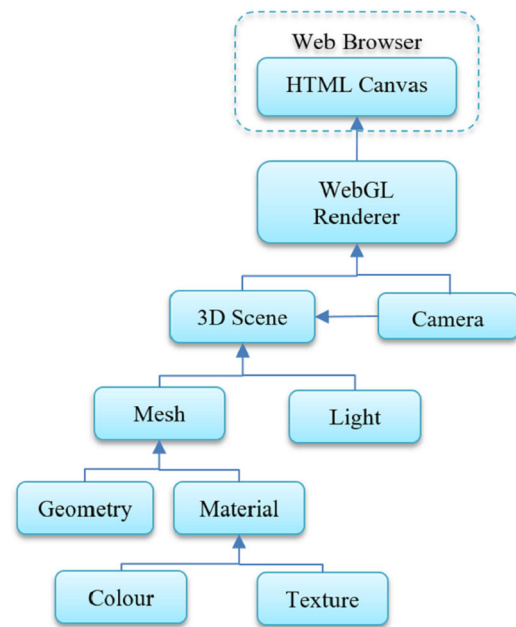


FIGURE 1 Sequential steps for creating any 3D geometry within the HTML5 canvas. 3D, three dimensional; WebGL, Web Graphics Library

The PUMA 560 robot model is built using this modeling technique, which is further used for its simulation and kinematic analysis. Each 3D component is assembled appropriately in a proper sequence to create the virtual 3D model of the PUMA 560 robot. Apart from the direct implication of predefined shapes, some constructive solid geometry can be made from the simple union, intersection principle of two shapes. This methodology of creating a 3D model from an assemblage of predefined geometry is implemented to develop the virtual model of another industrial robot RM501 Movemaster with 5 DOF, as shown in Figure 2. This model is shown here as an additional source of information that highlights the capabilities of this modeling technique. As stated earlier, the Movemaster robot is also used for teaching robot kinematics courses.

3.2 | Modeling using a CAD file import procedure

The process described above can be used for creating any 3D virtual model with simpler geometries having basic design ideas. In general, for components having an intricate design, it is better to use some CAD designing software for part design. The CAD file is then imported into the program for accurate modeling and visualization. As mentioned earlier, the direct use of the WebGL program is a little bit complicated as all the extracted data

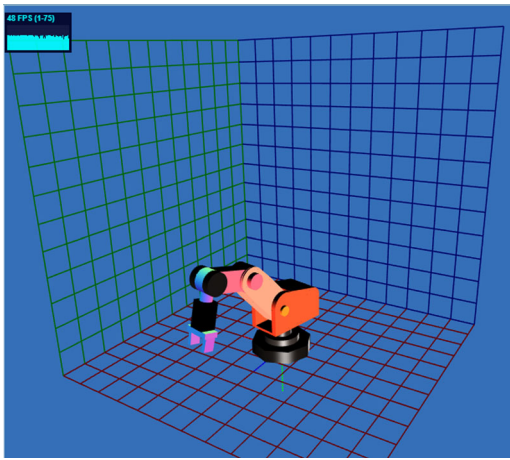


FIGURE 2 Movemaster RM501 robot 3D model. 3D, three dimensional

points from the CAD file need to be specified sequentially for exact resemblance. Therefore, here also using the library makes the process of CAD file import much easier and straightforward. To demonstrate this modeling procedure, the IIT Kharagpur KGP50 robot simulation model is created as it has multiple components with intricate shapes [11]. All the part components of the robot are initially designed in SolidWorks software, and then, the interface is prepared for CAD file import in the program script. The designed components are saved as stereolithography (STL) format and are loaded in the program through the Three.js STL loader library.

The sequential steps for importing any CAD model within the scene are shown in Figure 3. The initial stages of creating the HTML canvas and 3D scene remain the same, as stated earlier. As shown in Figure 3, the first part

is to create the part design in SolidWorks or using any other software and, then, converting the part file into STL format for file import. In the next step, the STL loader library is called in the program for visualizing the geometry. In this 3D modeling procedure, the vital step is the assembling of imported CAD files. Once a CAD geometry path is specified accurately in the program, a developer can visualize that geometry within the created HTML canvas. It can be positioned or oriented anywhere in the 3D scene by specifying its cartesian coordinates. Now assembling all part components in proper sequence can be done manually through translation or through applying a transformation matrix to the geometry. For perfect design, it is always recommended to add one geometry over another mathematically. The assembled 3D design of the IIT Kharagpur KGP50 robot, along with the actual robot model, is shown in Figure 3. The part design can also be saved in some other supporting format of Three.js apart from STL format. Some 3D building software like blender provides an extension of Three.js for 3D scene export, which helps in direct mapping into the web interface. As the entire content is uploaded in the global server for virtual labs program, as mentioned in the introduction, it also allows a user to download the CAD files for any other experimental purpose. The visualization level can further be increased by adding different material or image textures to the virtual model for actual resemblance.

3.3 | Simulation environment setup

Once the 3D model of the PUMA 560 robot is created within the scene, it is mapped with multiple other

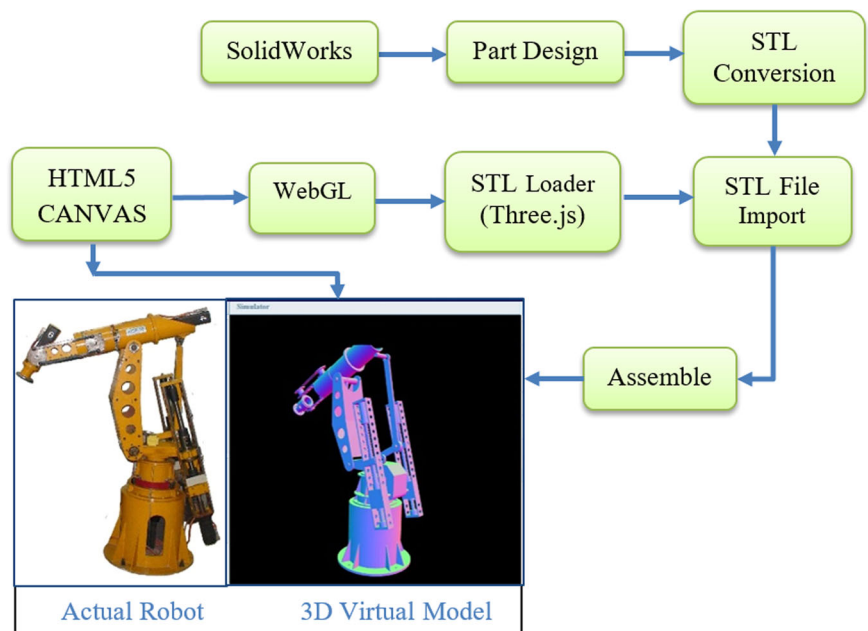


FIGURE 3 Modeling using a CAD file import within an HTML canvas. 3D, three dimensional; CAD, computer-aided design; STL, stereolithography; WebGL, Web Graphics Library

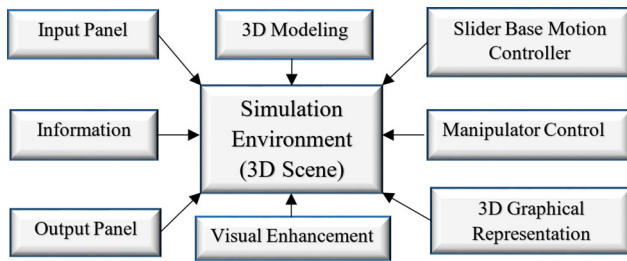


FIGURE 4 Components of the simulation interface. 3D, three dimensional

components of the interface for its kinematic study or to visualize any motion simulation. Here, for demonstration purposes, only fundamental features needed for the kinematic analysis of the robot are incorporated in the interface, which is linked with the 3D scene, as shown in Figure 4. It can be extended up to any desired level depending on the developer's need, and almost any features can be added to the GUI. The 3D scene contains the virtual model of the robot prepared with any of the described 3D modeling techniques. The scene is then linked with other components of motion simulation, graph plot, and manipulator control along with input and output panel for kinematic study. Some other modules like information about the robot and visual enhancement procedure like magnification and rotation is also added to the 3D scene. As the view within the 3D environment can be altered at any orientation and the magnification level can be varied, an exact motion of the end-effector is tracked. The total canvas view can also be divided into dual or quad-port views to verify the movement of the robot from all projection angles. The whole content program is in JavaScript language and compacted in a single HTML file. The user needs to run the HTML file in any web browser with HTML5 enabled.

4 | THE KINEMATIC ANALYSIS OF PUMA 560 ROBOT

The two main aspects of the kinematic study of the PUMA 560 robot described in the theory section are demonstrated here from the interface. After creating the 3D virtual model in the HTML canvas div, the other sub-components of the interface shown in Figure 4 are programmed. Here, multiple JavaScript utility libraries were used for mathematical computation and simulation. The interface structure is changed as needed in the forward and inverse kinematic analysis, which can also be combined in a single interface. All these simulation modules are incorporated under each experiment in robotics

virtual labs of our institution, along with theory content and other supporting materials. A user needs to go through the mathematical aspect of the problem before experimenting with the simulation model for proper understanding.

4.1 | Forward kinematics and workspace analysis

In the forward kinematic analysis, input parameters are the joint angles of the robot. Therefore, in the input panel, joint angular data submission space is created. The robot default view is set in the zero angles kinematic configuration. Now by entering the joints values, after submission of data, two parallel changes occur in the interface output panel and the 3D scene. The robot is moved as per angular values, and its final configuration is seen in the 3D view. Simultaneously, the transformation matrix is also computed, which gives the position and orientation information of the robot end-effector from the base coordinate frame. The configuration of the PUMA 560 robot after the submission of two sets of data in the forward kinematic analysis is shown in Figure 5. The individual joint movement within the robot working range is observed by moving the slider of each joint from the controller panel of the interface. The end-point positional data of the manipulator at a particular configuration can also be obtained by clicking the "Click" button in the controller tab.

Another important aspect of study in context to this is the workspace analysis of any robotic manipulator. The reachable workspace is the total volume of space that the manipulator end-effector can reach within its joint range. Therefore, to visualize the position in 3D space, a graphical view panel is created within the interface using a library of plotly.js. In this graphical panel, the point graph is plotted with each robot movement, which gives the position information of the robot end-effector from the base frame. The workspace of the PUMA robot is generated from the "workspace" tab in the controller panel of the interface, which is shown separately in Figure 6. The format of the plotting chart can be changed from point plotting to line graph, which shows the motion trajectory of the robot. It helps in planning the trajectory from the simulation model. The interface also allows saving the graphical data in the CSV file format.

4.2 | Inverse kinematics analysis

The forward kinematics formulation is easy to set up through simple matrix multiplication as there is always

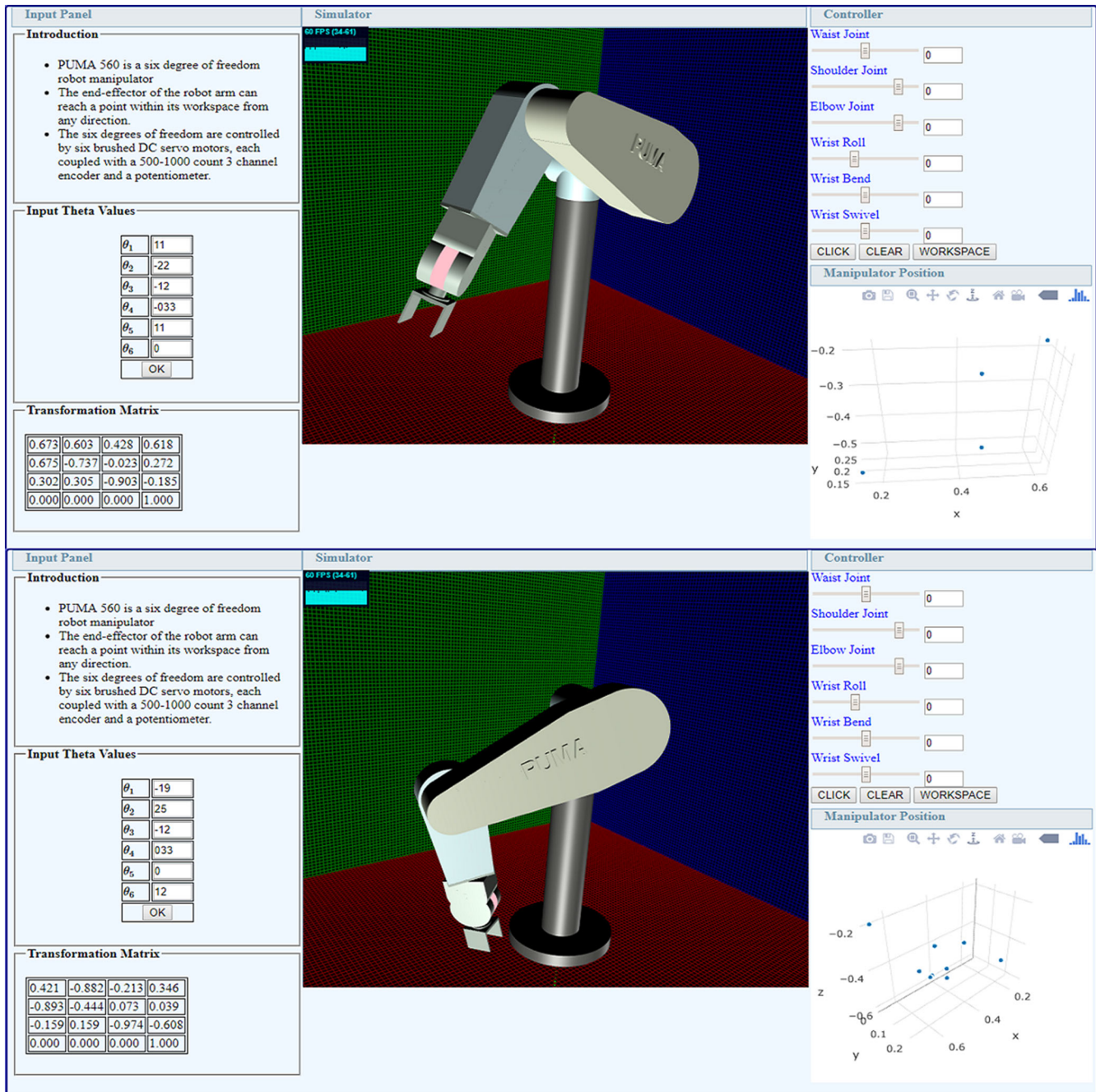


FIGURE 5 Forward kinematic analysis of the PUMA 560. PUMA, programmable universal machine for assembly

a solution for the forward kinematic problem. For the inverse kinematics problem, a complex algebraic mathematical computation is needed to get all the possible solutions. The first criteria that need to be checked are whether the specified desired position lies within the workspace region of the robot. Now depending on the number of joints and link parameters, there can be multiple solutions for a single goal. The PUMA 560 robot can reach some positions with up to eight different joint solutions. An additional constraint comes from joint range limitations, which reduce the accessibility of

all solutions. Here, for visualization purposes, all the possible solutions are derived in the interface, out of which some configurations may not be feasible in the actual case due to this joints limitation. The inverse kinematics analysis is carried out by changing the interface input panel structure where the user can specify the position to reach with a particular orientation of the end-effector. On each submission of data, mathematical computation is done using math.js library embedded in a JavaScript file. It computes the joint parameters using the Equations (5)–(15). In the output section, it shows

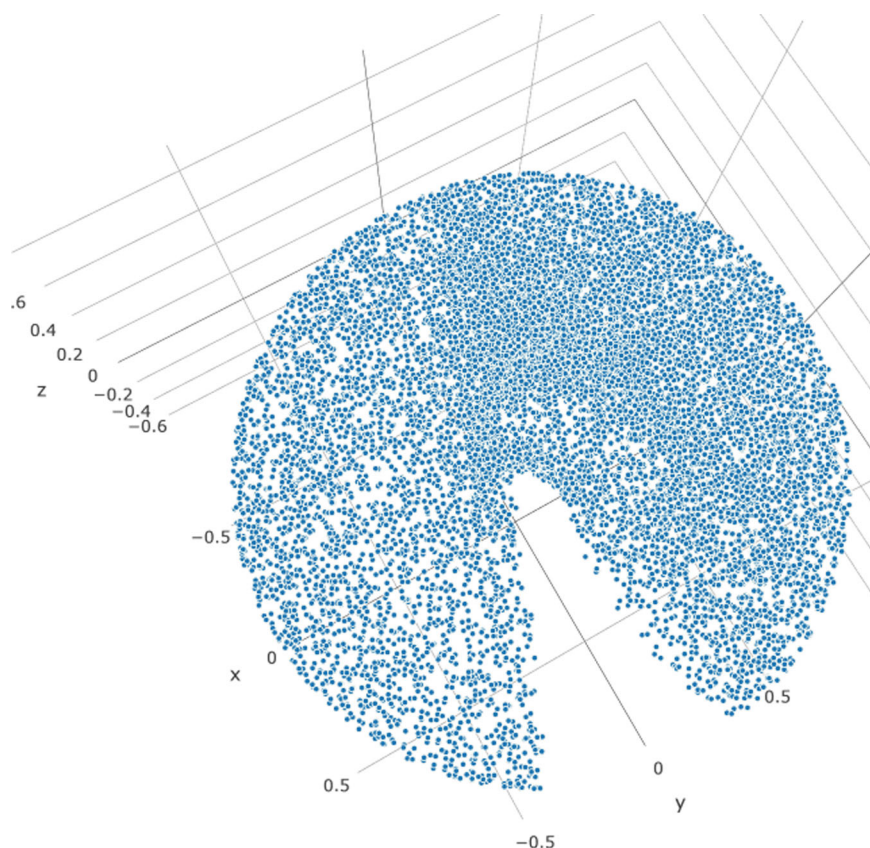


FIGURE 6 The workspace of the PUMA 560 robot. PUMA, programmable universal machine for assembly

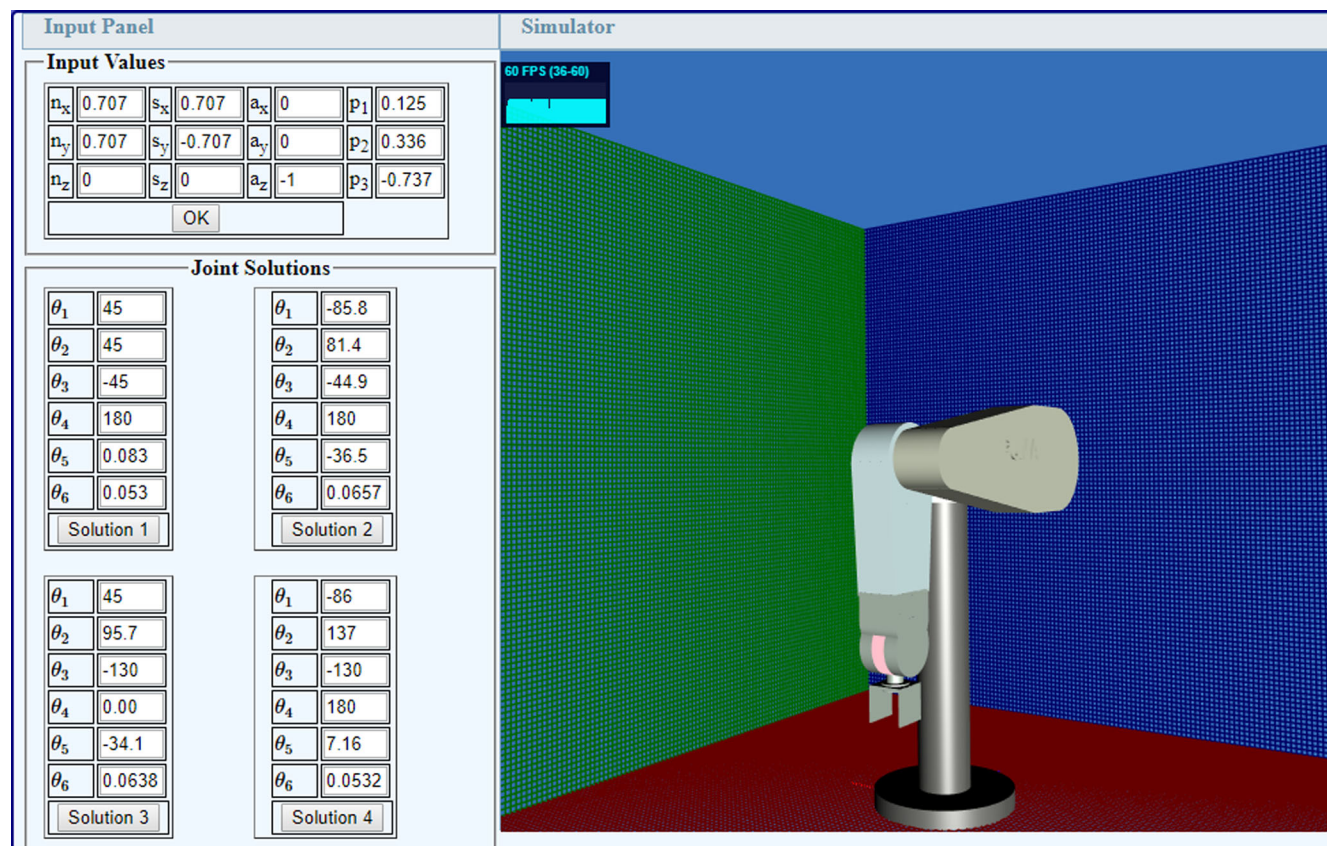


FIGURE 7 Interface for Inverse kinematics analysis

the possible solutions for the particular configuration data input from the user. Figure 7 shows the inverse kinematics simulation environment with a default configurational 3D View of the PUMA 560 robot.

Each configuration of the manipulator is visualized in the 3D environment by a mouse button click at each solution highlighted "solution i " tab. The configurations for a specified position and orientation of the robot are shown in Figure 8. As mentioned in earlier, there can be a set of total eight joint solutions for an existing configuration, out of which four are just wrist flip solution which can be easily found out from the following relations:

$$\theta'_4 = \theta_4 + 180^\circ, \quad (16)$$

$$\theta'_5 = -\theta_5, \quad (17)$$

$$\theta'_6 = \theta_6 + 180^\circ. \quad (18)$$

5 | REMOTE CONTROL OF PUMA 560 FROM THE INTERFACE

The kinematic analysis performed with the virtual robot can be verified with an actual robot through its remote operation from the interface. The control techniques of any actual robotic system have been proposed earlier, following different communication procedures for learning through remotely triggered laboratories [30,31,44]. The remotely triggered laboratories for robotics are developed with the concept of telepresence, where a vision system is used for the visualization of actual robotic movements. Here, as the total content is in web format, a web-based communication architecture is set up, which does not require the installation of any software or plugins for mathematical computations on the server-side for executing the program.

A local webserver is first created using WAMP (Windows, Apache, MySQL, and PHP), and the whole

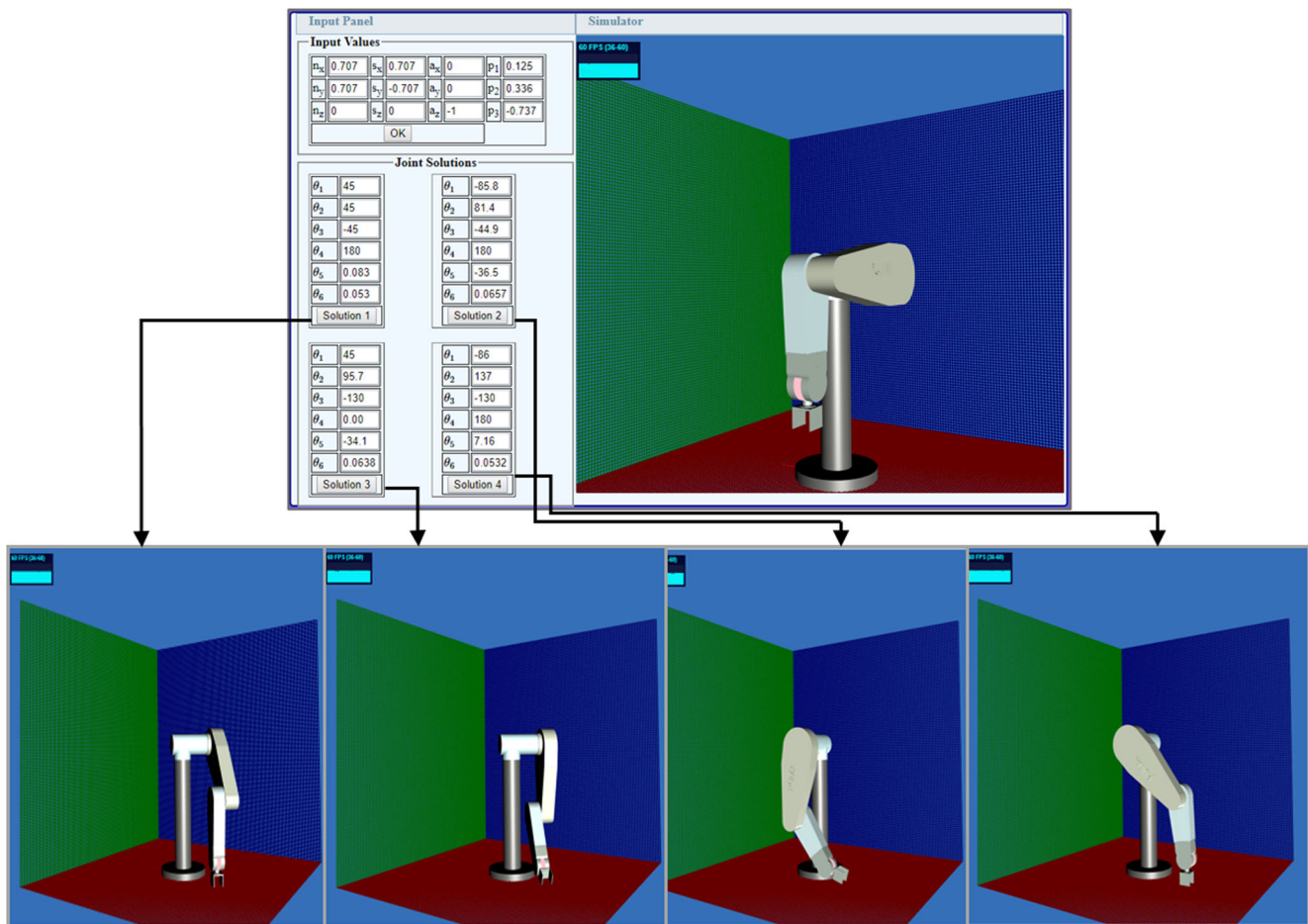


FIGURE 8 Multiple solutions visualization of the PUMA 560 robot in inverse kinematics analysis. PUMA, programmable universal machine for assembly

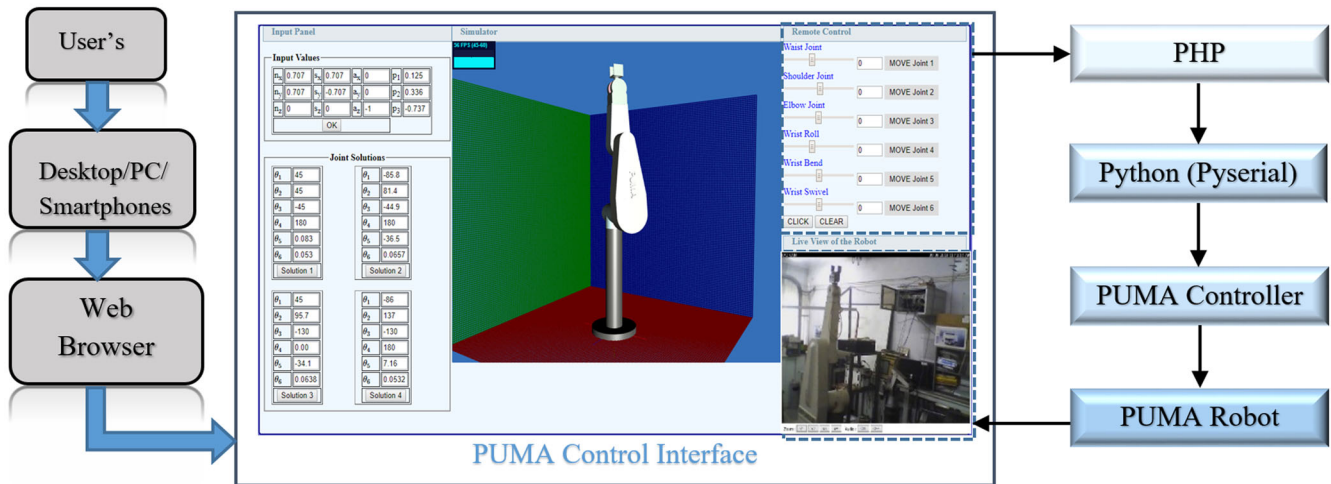


FIGURE 9 PUMA 560 robot control from the interface with direct visual feedback. PC, personal computer; PUMA, PUMA, programmable universal machine for assembly

program is uploaded into that local server through which its communication with the actual robot controller is initially set up. The source code is later deployed into the global server, and a client-server communication is established in the python level through TCP/IP protocol. The structure for remote operation of the PUMA 560 robot from the developed GUI is shown with the block diagram in Figure 9. It shows the sequential steps of communication architecture for robot joint actuation. The procedure shown here is quite simple and does not require any intermediate supporting application for executing the program. A robot joint control method is added in the interface as a "Manipulator Control" panel where slider based data input and a button-based control structure created for actuating the robot. The serial communication port of the PUMA controller is assessed through the Python programming language. Python is an interpreter-based language, and it has multiple modules that are easily embedded into the PHP programming part for any web-based communication system. On each joint movement command from the interface control panel directly runs the python scripts through PHP 'exec' execution command. The Python scripts are developed using the *pyserial* module, which opens the serial communication port and reads or writes the data to the PUMA controller. The direct visual feedback is obtained through a vision system incorporated within the experimental setup. The live view of the actual PUMA robot is received in the interface from an IP camera, which monitors the robot movements. A user can get parallel visualization from the virtual model and the actual robot. This type of online system access for learning is most useful in group demonstration. However, it limits

individual access for students at the same time. The system response time is also one of the significant factors, which needs to be taken into consideration while designing the communication setup. Therefore, it is mostly preferred to understand the basic concepts of robot kinematics and joint actuation through simulations and offline training before controlling the actual system.

6 | CONCLUSION

The necessity of open-source and application-independent platforms is increasing rapidly for better applicability in an online education system. It reduces the dependencies on the server or client, which enhances the E-learning/education process. The work described here provides an efficient solution to the implementation of any 3D simulations on a web page for enhanced learning purposes. This type of web-format simulation platform is mostly needed for the development of virtual laboratories. The content is independent of any plugins or applications like java, flash, and so forth; therefore, the simulations can run from the smartphone also. The application of WebGL technology implied for 3D modeling of robotic manipulators can be used to develop any 3D model within an HTML canvas. Using any utility library simplifies the WebGL program and provides additional features for enhanced visualization. The web interface designed for PUMA robot analysis is connected with an actual system for its remote control and verification of experiments. It also helps a student in a better understanding of experimental simulations through parallel visualization of the virtual and actual models.

The framework shown here for building the interface of the PUMA 560 robot can be implied to any other robotic system for its kinematic study. The work will be further extended for dynamic analysis of the PUMA 560 robot by adding physics to the scene. For better applicability with the real robot variable assembly language (VAL), programming-based motion control will be included further in the simulation interface. It will help a student to develop an idea about the robot programming script. The application of this type of web format interface is not limited to only industrial robotics analysis. It can be applied to any other field of robotics research like humanoid robotics, swarm robotics, and medical applications. The concept of multiple solution visualization within the same environment demonstrated here can also be applied for multiple robot motion simulation within the same view. This type of platform is also useful for research-based work on robotics through virtual reality interaction.

ACKNOWLEDGMENTS

The authors gratefully acknowledge the support of MHRD, New Delhi, and Department of Mechanical Engineering, Indian Institute of Technology, Kharagpur for providing the platform to carry out the work.

ORCID

Ujjal Dey  <http://orcid.org/0000-0002-0382-6225>

REFERENCES

1. M. S. Alshamasin et al., *Kinematic modeling and simulation of a scara robot by using solid dynamics and verification by matlab/simulink*, Eur. J. Sci. Res. **37** (2009), no. 3, 388–405.
2. B. Balamuralithara and P. C. Woods, *Virtual laboratories in engineering education: The simulation lab and remote lab*, Comput. Appl. Eng. Educ. **17** (2009), no. 1, 108–118.
3. F. A. Candelas et al., *A virtual laboratory for teaching robotics*, Complexity **1** (2003), 10–11.
4. Y. Cao, *Learning robotics through developing a virtual robot simulator in MATLAB*. Washington, DC: American Society for Engineering Education, 2011.
5. D. Chaos et al., *Virtual and remote robotic laboratory using EJS, MATLAB and LabVIEW*, Sensors **13** (2013), no. 2, 2595–2612.
6. P. I. Corke, *A robotics toolbox for MATLAB*, IEEE Robot. Autom. Mag. **3** (1996), no. 1, 24–32.
7. P. I. Corke, *MATLAB toolboxes: Robotics and vision for students and teachers*, IEEE Robot. Autom. Mag. **14** (2007), no. 4, 16–17.
8. J. J. Craig, *Introduction to robotics: Mechanics and control*, 3rd ed., Pearson Education India, Bengaluru, India, 2009.
9. T. De Jong, M. C. Linn and Z. C. Zacharia, *Physical and virtual laboratories in science and engineering education*, Science **340** (2013), no. 6130, 305–308.
10. J. Denavit and R. Hartenberg, *A kinematic notation for low pair mechanisms based on matrices*, ASME J. Appl. Mech. **22** (1955), 215–221.
11. U. Dey and C. S. Kumar, *Kinematic analysis and simulation of a 6 DOF robot in a web-based platform using CAD file import*, Proc. Int. Conf. Intell. Syst. Des. Appl., 2017.
12. U. Dey, K. J. Pabitra and C. S. Kumar, *Modeling and kinematic analysis of industrial robots in WebGL interface*, Proc. IEEE 8th Int. Conf. Technol. Educ. (T4E 2016), 2016.
13. J. Dirksen, *Learning Three.js: The JavaScript 3D library for WebGL*, Packt Publishing Ltd., Birmingham, UK, 2013.
14. W. E. Dixon et al., *A simulink-based robotic toolkit for simulation and control of the PUMA 560 robot manipulator*, Proc. 2001 IEEE/RSJ Int. Confe. Intell. Robots Syst., Expanding Societal Role Robot. Next Millennium (Cat. No. 01CH37180), vol. 4, 2001.
15. R. Falconi and C. Melchiorri, *Robotcad: An educational tool for robotics*, IFAC Proc. Vol. **41** (2008), no. 2, 9111–9116.
16. V. Fedák, Ď. František and Ľ. Róbert, *MATLAB applications for the practical engineer*, Analysis of robotic system motion in SimMechanics and MATLAB GUI environment, IntechOpen, London, UK, 2014. IntechOpen.
17. F. Fonseca, M. Nuno and D. C. F. Elisabete, *Computer applications for education on industrial robotic systems*, Comput. Appl. Eng. Educ. **26** (2018), no. 5, 1186–1194.
18. M. Freese et al., *Virtual robot experimentation platform V-REP: A versatile 3D robot simulator*, Int. Conf. Simul., Model., Programm. Auton. Robots, 2010.
19. M. Gouasmi et al., *Kinematic modelling and simulation of a 2-R robot using solidworks and verification by MATLAB/Simulink*, Int. J. Adv. Robot. Syst. **9** (2012), no. 6, 245.
20. S. Ivaldi, V. Padois and F. Nori, *Tools for dynamics simulation of robots: a survey based on user feedback*, arXiv (2014). <https://arxiv.org/abs/1402.7050>
21. C. A. Jara et al., *EJS+EjsRL: An interactive tool for industrial robots simulation, computer vision and remote operation*, Rob. Auton. Syst. **59** (2011), no. 6, 389–401.
22. C. Kefalis and A. Drigas, *Web based and online applications in STEM education*, Int. J. Eng. Pedagogy **9** (2019), no. 4, 76–85.
23. L. Lianzhong, Z. Kun and X. Yang, *A cloud-based framework for robot simulation using WebGL*, Proc. 6th Int. Conf. Intell. Syst. Des. Eng. Appl., 2015.
24. J.-J. Luo and H. U. Guo-Qing, *Study on the simulation of robot motion based on MATLAB*, J. Xiamen Univ. Natur. Sci. **44** (2005), no. 5, 640.
25. S. Manzoor et al., *An open-source multi-DOF articulated robotic educational platform for autonomous object manipulation*, Robot. Comput. Integr. Manuf. **30** (2014), no. 3, 351–362.
26. C. Marcu, G. Lazea and R. Robotin, *An OpenGL application for industrial robots simulation*, Proc. 2006 IEEE Int. Conf. Autom., Qual. Testing, Robot., vol. 2, 2006.
27. C. McCormack and D. Jones, *Building a web-based education system*, John Wiley & Sons, Inc, Hoboken, NJ, 1997.
28. D. Miller, I. Nourbakhsh and R. Siegwart, *Robots for education in the Springer handbook of robotics*, Springer, Berlin, Germany, 2008, pp. 1283–1301.
29. R. M. Murray, *A mathematical introduction to robotic manipulation*, CRC press, Boca Raton, FL, 2017.
30. A. T. Navaraja et al., *CAD/CAM, robotics and factories of the future*, Web based simulation and remote triggered laboratory for robots, Springer, New Delhi, India, 2016, pp. 665–677.
31. S. Osentoski et al., *Remote robotic laboratories for learning from demonstration*, Int. J. Social Robot. **4** (2012), no. 4, 449–461.

32. R. S. Othayoth et al., *Robot kinematics made easy using RoboAnalyzer software*, Comput. Appl. Eng. Educ. **25** (2017), no. 5, 669–680.
33. T. Parisi, *Programming 3D applications with HTML5 and WebGL: 3D animation and visualization for web pages*, O'Reilly Media, Inc., Sebastopol, CA, 2014.
34. M. Quigley et al., *ROS: An open-source robot operating system*, Proc. ICRA Workshop Open Source Software, vol. 3, no. 2, 2009.
35. E. Rohmer, P. N. S. Surya and M. Freese, *V-REP: A versatile and scalable robot simulation framework*, Proc. 2013 IEEE/RSJ Int. Conf. Intell. Robots Syst., 2013.
36. V. Román-Ibáñez et al., *A low-cost immersive virtual reality system for teaching robotic manipulators programming*, Sustainability **10** (2018), no. 4, 1102.
37. D. Sengupta, N. Jain and C. S. Kumar, *An educational website on kinematics of robots*, Proc. IEEE 5th Int. Conf. Technol. Educ. (T4E 2013), 2013.
38. O. Severa, M. Goubej and J. Konigsmarkova, *Unified framework for generation of 3D web visualization for mechatronic systems*, J. Phys., Conf. Ser. **659** (2015), no. 1, 012053.
39. Y. Shaoqiang, L. Zhong, and L. Kingshan, *Modeling and simulation of robot based on Matlab/SimMechanics*, Proc. IEEE 27th Chin. Control Conf., 2008.
40. M. Szilvsi-Nagy and G. Y. Matyasi, *Analysis of STL files*, Math. Comput. Model. **38** (2003), no. 7–9, 945–960.
41. G. Tavares, *WebGL fundamentals*, HTML5 Rocks 250, 2012.
42. L.-W. Tsai, *Robot analysis: The mechanics of serial and parallel manipulators*, John Wiley & Sons, Hoboken, NJ, 1999.
43. B. Tudjarov, J. Botzheim and N. Kubota, *Facilitation of cognitive robotics by web based computational intelligent models*, Proc. IEEE Int. Symp. Micro-NanoMechatron. Hum. Sci. (MHS 2012), 2012.
44. S. G. Tzafestas, *Web based control and robotics education*, Springer, Berlin, Germany, 2009.
45. X. Xu et al., *Robotic kinematics teaching system with virtual reality, remote control and an on-site laboratory*, Int. J. Mech. Eng. Educ. **48** (2018), no. 3, 197–220.
46. C. Yang et al., *Modeling and simulation of spatial 6-DOF parallel robots using Simulink and SimMechanics*, Proc. IEEE 3rd Int. Conf. Comput. Sci. Inf. Technol., vol. 4, 2010.
47. Z. Zhang et al., *Development of an OpenGL based multi-robot simulating platform*, Proc. IEEE 8th Control, Autom., Robot. Vis. Conf. (ICARCV 2004), vol. 2, 2004.
48. D. Zhou et al., *A teaching method for the theory and application of robot kinematics based on MATLAB and V-REP*, Comput. Appl. Eng. Educ. **28** (2020), 239–253.

AUTHOR BIOGRAPHIES



Ujjal Dey completed his BTech degree in Mechanical Engineering from West Bengal University of Technology (India) in 2013 and MS (Research) degree from the Indian Institute of Technology (IIT), Kharagpur in 2019. He has research experience from Indian Institute of Engineering Science and Technology (IIEST), Shibpur and IIT Kharagpur in the field of Orbital Mechanics, Newtonian Gravitation, and Robotics. He is currently a PhD research scholar from the Department of Mechanical Engineering at the Indian Institute of Technology Kharagpur. His research interests include robotics, micro-nano manipulation, CAD, virtual reality.



Kumar Cheruvu S. received his Bachelors of Technology degree in Manufacturing Science and Engineering in 1987 and his Doctoral degree in Mechanical Engineering in 1995 from the Indian Institute of Technology Kharagpur. He was a post-doctoral fellow of the Science and Technology Agency (now JSPS) at AIST (formerly Electro Technical Laboratory), Tsukuba, Japan in 1999. His research areas span Robotics: Control of Autonomous Underwater Vehicles, Flexure based micro-grippers, micro and nano robotics, humanoid robots, biomedical rehabilitation robotics; Direct Digital Manufacturing of Metals using additive processes and in computer networks.

How to cite this article: Dey U, Cheruvu SK. A web-based integrated GUI for 3D modeling, kinematic study, and control of robotic manipulators. *Comput Appl Eng Educ*. 2020;28:1028–1040.
<https://doi.org/10.1002/cae.22282>